

Match Mate

REST API Documentation

Property	Value
Version	1.0-SNAPSHOT
Framework	Spring Boot 4.0.0
Java	Java 21
Database	PostgreSQL + Liquibase
Auth	JWT (JJWT 0.11.5)
Base URL	/api/v1
Package	com.edu.basic

Version 1.0-SNAPSHOT • June 2026

1. Overview

The Basic Resource Package is a Spring Boot REST API for a resource booking and event management platform. It exposes endpoints for user authentication, event and booking management, and payment handling. The API is secured with JWT tokens and uses PostgreSQL as the primary database with Liquibase for schema versioning.

1.1 Technology Stack

Technology	Version / Detail	Description
Spring Boot	4.0.0	Core framework, dependency injection, auto-configuration
Spring Security	6.x	Authentication and authorization with JWT
Spring Data JPA	3.x	ORM layer over PostgreSQL
Liquibase	Latest	Database schema migrations (db/changelog/changes/)
JJWT	0.11.5	JWT generation, parsing, and validation
Lombok	1.18.42	Boilerplate reduction (@Data, @Builder, @Slf4j, etc.)
PostgreSQL	Runtime	Primary relational database
Actuator	Included	Health checks and monitoring endpoints

1.2 Package Structure

The project follows a domain-driven layered architecture under com.edu.basic:

Package / File	Type	Description
booking/	Package	Booking controllers, services, and DTOs
config/	Package	Spring Security, CORS, and app configuration beans
entity/	Package	JPA entity classes mapped to DB tables
event/	Package	Event management controllers and services
events/	Package	Domain events (application event publishing)
exception/	Package	Global exception handler and custom exception classes
payment/	Package	Payment processing controllers and services
repository/	Package	Spring Data JPA repositories (note: typo in package name)
Security/	Package	JWT filter, token provider, and UserDetailsService
service/	Package	Shared/common service layer
user/	Package	User registration, profile, and authentication
Main.java	Class	Spring Boot entry point (@SpringBootApplication)

1.3 Database Schema

Liquibase manages the schema through four ordered changesets in db/changelog/changes/:

File	Type	Description
01-create-user-table.xml	Migration	Users table – credentials, roles, profile fields
02-create-event-tables.xml	Migration	Events table – title, description, capacity, dates
03-create-bookings-table.xml	Migration	Bookings table – user-event link, status, timestamps
04-create-payments-table.xml	Migration	Payments table – amount, method, booking FK, status

2. Getting Started

2.1 Prerequisites

- Java 21 (LTS)
- Maven 3.8+
- Docker & Docker Compose (recommended for PostgreSQL)
- PostgreSQL 15+ (if not using Docker)

2.2 Running with Docker

The project ships with a docker-compose.yml for one-command local setup:

```
# Start database
docker compose up -d

# Run application
mvn spring-boot:run -Dspring-boot.run.profiles=dev
```

2.3 Configuration Files

- application.yml — production/default configuration.
- application-dev.yml — local development overrides (typically points to localhost:5432).

Activate dev profile with:

```
SPRING_PROFILES_ACTIVE=dev # env variable
# or
spring.profiles.active=dev # application.yml property
```

2.4 Key Configuration Properties

Property	Type	Description
spring.datasource.url	String	JDBC URL — jdbc:postgresql://host:5432/dbname
spring.datasource.username	String	PostgreSQL username
spring.datasource.password	String	PostgreSQL password
spring.liquibase.change-log	String	classpath:db/changelog/db.changelog-master.xml
jwt.secret	String	Base64-encoded HMAC secret (min 256-bit recommended)
jwt.expiration	Long	Token TTL in milliseconds (e.g., 86400000 = 24 h)
spring.jpa.hibernate.ddl-auto	String	Set to none — Liquibase owns schema management
server.port	Integer	Default 8080; override per environment

3. User Management API

The User Management module handles user authentication, registration, profile management, and user information retrieval.

Base URL: /v1

Authentication: Most user-related APIs require JWT authentication.

`Authorization: Bearer <JWT_TOKEN>`

3.1 Authentication Controller

3.1.1 User Registration

Endpoint: POST /v1/auth/register

Creates a new user account and returns an authentication token along with user details.

Field	Type	Description
email	String	User email address
password	String	User password
firstName	String	User first name
lastName	String	User last name
age	Integer	User age
gender	String	User gender
phoneNumber	String	Contact number
city	String	User city
country	String	User country

3.1.2 User Login

Endpoint: POST /v1/auth/login

Authenticates an existing user and generates a JWT authentication token.

Field	Type	Description
email	String	Registered user email
password	String	User password

3.1.3 Check Email Availability

Endpoint: GET /v1/auth/check-email/{email}

Checks whether an email address is already registered.

3.2 User Controller

3.2.1 Get User Profile

Endpoint: GET /v1/users/{userId}

Retrieves user profile information using user ID.

3.2.2 Update User Profile

Endpoint: PUT /v1/users/{userId}

Updates existing user profile information.

Field	Type	Description
firstName	String	Updated first name
lastName	String	Updated last name
age	Integer	Updated age
gender	String	Updated gender
phoneNumber	String	Updated phone
bio	String	User description
city	String	Current city
country	String	Current country
profileImageUrl	String	Profile image URL

3.2.3 Delete User Account

Endpoint: DELETE /v1/users/{userId}

Deletes a user account permanently.

3.3 Database Entity: User

Table Name: users

Column	Type	Description
user_id	Long	Primary Key
email	VARCHAR(100)	Unique user email
password	VARCHAR	Encrypted password
first_name	VARCHAR(50)	First name
last_name	VARCHAR(50)	Last name
age	Integer	User age
gender	VARCHAR(20)	User gender
phone_number	VARCHAR(20)	Contact number
bio	TEXT	User biography
role	VARCHAR(20)	User role
is_active	Boolean	Account status
is_email_verified	Boolean	Email verification status
city	VARCHAR(100)	User city
country	VARCHAR(100)	User country

Column	Type	Description
profile_image_url	VARCHAR(255)	Profile image URL

3.4 Architecture Components

- Controller Layer: Handles HTTP requests, validation, and responses. (AuthController, UserController)
- Service Layer: Business logic, authentication processing, and user management. (AuthServiceImpl, UserServiceImpl)
- Repository Layer: Database communication. (UserRepository)
- Mapper Layer: Entity ↔ DTO conversion. (UserMapper)

4. Event Management API

The Event Management module is the core engine for organizing dating events. It facilitates the end-to-end lifecycle of an event, from creation and participant management to geographic discovery and status tracking.

4.1 Module Overview

This module provides high-performance endpoints for:

- **Creation & Lifecycle:** Defining new events and managing their transition through various states (Upcoming, Ongoing, Completed).
- **Discovery:** Searching, filtering by type/status, and location-based discovery via geographic coordinates.
- **Capacity Management:** Handling real-time seat availability and gender-balanced participant limits.
- **Administrative Control:** Granular updates for authorized creators and system administrators.

Base URL: `{{host}}/v1/events`

4.2 Common API Response Format

All event-related endpoints utilize a standardized response wrapper:

```
{
  "success": true,
  "message": "Operation description",
  "data": {},
  "statusCode": 200
}
```

Field	Type	Description
success	Boolean	Indicates if the operation was successful.
message	String	Contextual feedback for the user or developer.
data	Object	The primary payload (DTO or List).
statusCode	Integer	Standard HTTP status code.

4.3 Data Transfer Objects (DTOs)

4.3.1 EventRequestDTO

Used exclusively for creating or updating event resources.

Field	Type	Required	Description
eventName	String	Yes	Min 3, Max 100 characters.
description	String	Yes	Min 10, Max 500 characters.
eventType	Enum	Yes	SPEED_DATING, GROUP_DATING, etc.
eventDate	LocalDateTime	Yes	Must be a future date/time.

Field	Type	Required	Description
eventEndDate	LocalDateTime	Yes	Must be after the start date.
location	String	Yes	Venue address or name.
latitude	Double	Yes	Between -90 and 90.
longitude	Double	Yes	Between -180 and 180.
ticketPrice	BigDecimal	Yes	Must be greater than 0.
totalSeats	Integer	Yes	Between 1 and 10,000.
imageUrl	String	No	URL to the event's promotional image.
specialInstructions	String	No	Max 1000 characters.

4.3.2 EventResponseDTO

The standard structure returned for all event retrieval requests.

Field	Type	Description
eventId	Long	Unique identifier.
eventStatus	EventStatus	UPCOMING, ONGOING, COMPLETED, CANCELLED.
availableSeats	Integer	Calculated as totalSeats - confirmed participants.
bookingPercentage	Integer	Dynamic progress indicator.
confirmedMaleCount	Integer	Current male participant count.
confirmedFemaleCount	Integer	Current female participant count.
createdByUsername	String	Email of the event organizer.

4.4 API Endpoints

4.4.1 Event Creation & Management

#	Method	Endpoint	Auth	Description
1	POST	/create	Bearer JWT	Creates a new event and assigns the authenticated user as the creator.
2	PUT	/{"eventId"}	Bearer JWT	Allows the creator to modify event details. Throws 403 Forbidden if user is not the creator.
3	DELETE	/{"eventId"}	Bearer JWT	Permanently removes the event. Restrictive to the creator.

4.4.2 Discovery & Search

#	Method	Endpoint	Params	Description
4	GET	/nearby	latitude, longitude, radius (Default: 50km)	Geospatial search to find events within a specific radius of coordinates.
5	GET	/search	keyword (String)	Partial matches on event names.
6	GET	/status/{status}	—	Filter by status: UPCOMING, ONGOING, etc.
7	GET	/type/{type}	—	Filter by type: DINNER_NIGHT, ADVENTURE_DATE, etc.

4.4.3 Participant & Admin Controls

#	Method	Endpoint	Auth	Description
8	PUT	/event/{eventId}/limits	ROLE_ADMIN	Update maleLimit and femaleLimit. New limits cannot be lower than existing confirmed bookings.

4.5 Business & Validation Rules

1. Temporal Integrity: An event's end date must always follow its start date.
2. Ownership: Only the createdByUserId has permission to modify or delete a specific event record.
3. Default State: Every new event is initialized with the UPCOMING status.
4. Automatic Scaling: Adjusting gender limits via the Admin API automatically recalculates the totalSeats and availableSeats fields.
5. Data Integrity: Gender-based limits are strictly enforced; limits cannot be decreased below the number of users already registered for that event.

4.6 Database Schema — Event Entity

Column	Type	Constraints
event_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT
event_name	VARCHAR(100)	NOT NULL
event_status	ENUM	UPCOMING, ONGOING, COMPLETED, CANCELLED
ticket_price	DECIMAL(19,2)	NOT NULL, CHECK > 0
male_limit	INTEGER	Default 0
female_limit	INTEGER	Default 0
created_by	BIGINT	FOREIGN KEY (users.id)

5. Payment Management API

The Payment Management module handles online payment processing for event bookings using the PayHere payment gateway.

This module provides functionality for:

- Initiating payments
- Generating PayHere payment hash
- Receiving PayHere payment callbacks
- Verifying payment authenticity
- Updating payment status
- Automatically confirming bookings after successful payments
- Retrieving payment details by booking ID

Base URL: `/v1/payments`

5.1 Payment Flow Overview

```
User
|
| Create Booking
|
| Booking Status = PENDING
|
| Initiate Payment API
|
| Generate PayHere Hash
|
| PayHere Payment Gateway
|
| Payment Callback
|
| Verify MD5 Signature
|
| Update Payment Status
|
v Confirm Booking
```

5.2 Payment Status Flow

From Status	To Status	Trigger
PENDING	COMPLETED	Successful PayHere callback (status_code = 2)
COMPLETED	Booking CONFIRMED	Automatic on payment completion
PENDING	FAILED	Cancelled / Failed / Chargeback callback

5.3 Authentication

Endpoint	Auth Required	Reason
POST /v1/payments/initiate	Yes (JWT Bearer)	Protected – only booking owner can initiate.
POST /v1/payments/callback	No	PayHere servers call this endpoint directly after payment.

Authorization: Bearer <JWT_TOKEN>

5.4 DTO Documentation

5.4.1 PaymentRequest

Used to initiate a payment.

```
{  
  "bookingId": 10,  
  "amount": 2500.00  
}
```

Field	Type	Required	Description
bookingId	Long	Yes	Booking identifier
amount	Double	Yes	Payment amount

Validation Rules:

Field	Rule
bookingId	Cannot be null
amount	Cannot be null

5.4.2 PaymentHashResponse

Returned when initiating payment. Contains required information for opening the PayHere payment popup.

```
{  
  "orderId": "MM-AB12CD34",  
  "amount": 2500.00,  
  "currency": "LKR",  
  "hash": "A8F23BC90...",  
  "merchantId": "123456"  
}
```

Field	Type	Description
orderId	String	Unique payment order ID
amount	Double	Payment amount
currency	String	Payment currency
hash	String	PayHere generated security hash
merchantId	String	PayHere merchant ID

5.4.3 PaymentResponse

Used to return payment information.

```
{
```

```
"paymentId": 5,
"bookingId": 10,
"orderId": "MM-AB12CD34",
"amount": 2500.00,
"currency": "LKR",
"status": "COMPLETED",
"createdAt": "2026-06-24T10:30:00"
}
```

Field	Type	Description
paymentId	Long	Payment record ID
bookingId	Long	Related booking ID
orderId	String	Payment order reference
amount	Double	Paid amount
currency	String	Payment currency
status	PaymentStatus	Current payment status
createdAt	LocalDateTime	Payment creation time

5.5 Payment Controller APIs

5.5.1 Initiate Payment

HTTP Method: POST

Endpoint: /v1/payments/initiate

Authentication: Required. User ID extracted from JWT via @AuthenticationPrincipal Long userId

Creates a payment record and generates a PayHere hash required for payment processing. Only the booking owner can initiate payment.

Request Body:

```
{
  "bookingId": 10,
  "amount": 3000.00
}
```

Success Response (200 OK):

```
{
  "orderId": "MM-A12345BC",
  "amount": 3000.00,
  "currency": "LKR",
  "hash": "7B91ACD...",
  "merchantId": "123456"
}
```

Business Rules — Validations Before Hash Generation:

6. Booking Exists: Booking ID must exist in the system.
7. Ownership Validation: The logged-in user must own the booking. (User A cannot pay for User B's booking.)
8. Booking Status Validation: Payment is allowed only when Booking Status = PENDING.
9. Duplicate Payment Prevention: If payment already completed, system returns: "Payment already completed for this booking".

Error Responses:

Scenario	Status Code	Message
Booking Not Found	404 NOT FOUND	Booking not found
Unauthorized Payment	401 UNAUTHORIZED	Unauthorized
Invalid Booking State	400 BAD REQUEST	Booking is not in PENDING state

5.5.2 PayHere Payment Callback**HTTP Method:** POST**Endpoint:** /v1/payments/callback**Authentication:** Not required — PayHere server communicates directly with backend.

Receives payment confirmation from PayHere server. This endpoint verifies the callback signature before updating payment status.

Request Parameters:

Parameter	Type	Description
merchant_id	String	PayHere merchant ID
order_id	String	Generated order ID
payment_id	String	PayHere payment ID
payhere_amount	String	Payment amount
payhere_currency	String	Currency
status_code	String	Payment result
md5sig	String	Verification signature

Example Callback:

```
merchant_id=123456
order_id=MM-AB12CD34
payment_id=987654
payhere_amount=3000.00
payhere_currency=LKR
status_code=2
md5sig=AABBCC123
```

PayHere Status Codes:

Status Code	Meaning	System Action
2	Payment successful	Complete payment + confirm booking
-1	Cancelled	Mark failed
-2	Failed	Mark failed
-3	Chargeback	Mark failed

Callback Security Validation (MD5 Signature):

```
MD5 (
```

```
    merchant_id +
    order_id +
    amount +
    currency +
    status_code +
    MD5(secret)
)

// Generated signature must match md5sig
// If validation fails → "Invalid MD5 signature"
```

Successful Callback Process:

- Payment Status → COMPLETED
- Booking Status → CONFIRMED

5.5.3 Get Payment By Booking ID

HTTP Method: GET

Endpoint: /v1/payments/booking/{bookingId}

Example: /v1/payments/booking/10

Response (200 OK):

```
{
  "paymentId": 5,
  "bookingId": 10,
  "orderId": "MM-AB12CD34",
  "amount": 2500,
  "currency": "LKR",
  "status": "COMPLETED"
}
```

5.6 Payment Entity — Database Structure

Table Name: payments

Column	Type	Description
id	Long	Primary key
booking_id	Long	Related booking
amount	Double	Payment amount
currency	String	Currency code
order_id	String	Unique payment reference
payhere_payment_id	String	PayHere transaction ID
hash	String	Security hash
status	Enum	Payment status
created_at	DateTime	Creation timestamp

Entity Relationship:

One Booking → One Payment (implemented using @OneToOne)

5.7 Repository Functions

Method	Description
<code>findByOrderId(String orderId)</code>	Find payment by order ID – used during callback processing.
<code>findByBooking_Id(Long bookingId)</code>	Find payment by booking – used to retrieve payment details.

5.8 Service Layer Responsibilities — PaymentServiceImpl

- Payment initialization
- PayHere hash generation
- Callback verification
- Payment status updates
- Booking confirmation
- Payment retrieval

5.9 Payment Security Features

Hash Generation

Before sending payment request, merchant data + secret key → MD5 Hash. Generated hash prevents payment manipulation.

Merchant Data + Secret Key → MD5 Hash

Callback Verification

Every PayHere callback is verified using MD5 signature. This prevents:

- Fake payment requests
- Unauthorized status updates
- Payment tampering

5.10 Complete Payment Module Flow

1. User creates booking
2. Booking Status = PENDING
3. User calls initiate payment API
4. Backend creates Payment record
5. Backend generates PayHere hash
6. User completes payment on PayHere
7. PayHere sends callback to backend
8. Backend validates MD5 signature
9. Payment Status = COMPLETED


```
|  
10. Booking Status = CONFIRMED
```